Finance Data Processing & Access Control Backend

🚀 Project Overview

This project is a backend system for a **Finance Dashboard Application**. It allows different users to manage and view financial data based on their roles.

The system is designed with a focus on:

- Clean architecture
- Role-based access control
- Scalable API design
- Maintainable and readable code

🛠️ Tech Stack

- **Node.js**
- **Express.js**
- **MongoDB (Mongoose)**
- **JWT Authentication**
- **bcrypt.js**

📁 Project Structure

```
finance-backend/
│
├── config/
├── models/
├── controllers/
├── routes/
├── middleware/
├── utils/
├── server.js
└── .env
```

🔐 Features

1. User & Role Management

- Create users
- Assign roles (Viewer, Analyst, Admin)
- Activate/Deactivate users
- Role-based access restrictions

2. Financial Records Management

- Create financial records
- View records
- Update records
- Delete records
- Filter support (basic)

3. Dashboard Summary APIs

- Total Income
- Total Expense
- Net Balance
- Aggregated financial insights

4. Access Control

- **Viewer** → Can view dashboard only
- **Analyst** → Can view records + dashboard
- **Admin** → Full access

5. Validation & Error Handling

- Input validation
- Proper status codes
- Meaningful error responses

⚙️ Setup Instructions

1. Clone Repository

```
git clone https://github.com/Niteshsharma831/Internship-
Task/tree/master/Zorvyn%20Tech/finance-backend
cd finance-backend
```

2. Install Dependencies

```
npm install
```

3. Configure Environment

Create a `.env` file:

```
MONGO_URI=your_mongodb_url
JWT_SECRET=your_secret_key
```

4. Run Server

```
npm run dev
```

```
nodemon server.js
```

Server will start at:

```
http://localhost:5000
```

📡 API Endpoints

🔑 Auth Routes

Method	Endpoint	Description
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user

💰 Record Routes

Method	Endpoint	Access
GET	/api/records	Admin, Analyst
POST	/api/records	Admin
PUT	/api/records/:id	Admin
DELETE	/api/records/:id	Admin

📊 Dashboard Routes

Method	Endpoint	Access
GET	/api/dashboard	All logged-in users

🔐 Authentication

- Uses **JWT Token**
- Pass token in headers:

```
Authorization: <your_token>
```

🧠 Design Decisions

- Modular architecture for scalability
- Separation of concerns (routes, controllers, models)
- Middleware-based role control
- Simple aggregation logic for dashboard
- Beginner-friendly structure with clean code

✨ Optional Improvements (Future Scope)

- Pagination & filtering
- Search functionality
- Advanced analytics
- Rate limiting
- Unit & integration testing
- API documentation (Swagger)

📌 Assumptions

- Roles are predefined (Viewer, Analyst, Admin)
- Authentication is handled via JWT
- Basic validation is implemented
- MongoDB is used for persistence

👨‍💻 Author

Nitesh Kumar Sharma 💼 Full Stack Developer | App Developer | UX/UI Designer

📧 Email: niteshkumarsharma831@gmail.com

📱 Contact: +91 9572861917

🌐 Portfolio: https://devcraftnitesh.vercel.app/